SC2002: Object Oriented Design and Programming

AY25/26 Semester 2 Group Assignment Report

Lab Group 6 (FAIA)

Github Repository:

https://github.com/GerrardRepo/SC2002-Turn-Based-Project.git

Name	Matriculation Number
Yee Woon Chun Gerrard	U2422647C
Yang Siqiao	U2520385A
Yang Danni	U2521576B
Zhong Xingyang	U2523765B

Table of Contents

1. **Introduction**
   - 1.1 Assumptions
   - 1.2 Game Structure
   - 1.3 Gameplay features
   - 1.4 Design consideration, application of OOP

2. **Application of SOLID Principles**
   - 2.1 Single Responsibility Principle
   - 2.2 Open Closed Principle
   - 2.3 Liskov Substitution Principle
   - 2.4 Interface Segregation Principle
   - 2.5 Dependency Inversion Principle

3. **Trade-offs**

4. **UML Class Diagram**

5. **UML Sequence Diagram**
   - 5.1 Gameplay Logic Flow

6. **Reflection**

7. **Declarations**

1. Introduction

This turn based combat game implements a single player RPG game where the user first selects a player to be either Warrior, or Wizard, and then fights through rounds of enemies determined by the level selected by the user.

This game implementation follows a layered architecture that follows closely to the boundary control and entity system.

Layer	**Classes**	**Role**
Boundary	GameCLI.java Main.java	The class between the user and the game. Collects inputs, passes it over to the control classes and displays the UI
Control	BattleEngine.java BattleSetup.java SpeedBasedTurnOrder.java	Handles the game flow,
Entity	All other classes	Low level classes for the game such as Warrior, Wolf, Goblin etc.

1.1 Assumptions

We have made a few assumptions while creating this game, as follows:

1. The whole game is single player, that is, the user only selects one player, either Warrior or Wizard, not both.

2. All combatants follow the same damage scheme, where it takes the higher of either 0 or attacker’s attack - target defence. Thus removing the possibility of negative damage dealt.

3. Cooldown duration of skills is always 3 rounds.

4.Enemy only attacks player, nothing else.

5. Exactly 2 items are given throughout the whole game, and its during setup, duplicates are allowed.

6. Backup only spawns once and it is exclusive to medium and hard levels only.

7.Combatant's turn order is determined by their speed which is bubblesorted.

1.2 Game Structure

The following graphs display the hierarchy of files in our game folders.

```
Combatant (Abstract)
    ├── Player (Extends)
    │       ├── Warrior
    │       └── Wizard
    └── Enemy (Extends)
            ├── Goblin
            └── Wolf
```

```
StatusEffect (Abstract)
    ├── StunEffect
    ├── DefendEffect
    └── SmokeBombInvulnerability
```

```
Action (Interface)
  ├── BasicAttack
  ├── Defend
  ├── ShieldBash
  ├── ArcaneBlast
  └── UseItem

Item (Interface)
  ├── Potion
  ├── PowerStone
  └── SmokeBomb

TurnOrderStrategy (Interface)
        └── SpeedBasedTurnOrder

EnemyActionStrategy (Interface)
          └── BasicAttackStrategy
```

1.3 Gameplay Features

Our game has incorporated several game features:

1.Dynamic Action menu
The action menu in the CLI only shows options that are available, ie. If a skill is on cooldown, the option to use the skill will not be shown

Before Using special skill Shield Bash for warrior:

```
--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item
4. Special Skill (Shield Bash)

Enter choice (1-4):
```

After using Shield Bash for warrior:

```
--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item

Enter choice (1-4):
```

Attempting to use Shield bash while its on Cooldown:

```
--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item

Enter choice (1-4): 4
That option is not available right now.
```

2. Auto Target Selection
If there is only one enemy left alive, the game auto selects the last enemy as the target instead of prompting an additional time.

User choosing target:

```
Enter choice (1-4): 1
Select target:
1. Goblin B (HP: 5/55)
2. Goblin C (HP: 55/55)

Enter choice (1-2): 1
Warrior -> performs BasicAttack on-> Goblin B: HP: 5 -> 0 (dmg: 40-15=25)
Goblin B has been ELIMINATED!
```

Game Auto selecting the last enemy:

```
--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item
4. Special Skill (Shield Bash)

Enter choice (1-4): 1
Target: Goblin C
Warrior -> performs BasicAttack on-> Goblin C: HP: 55 -> 30 (dmg: 40-15=25)
```

3. Defense Ceiling
We implemented a check for defense status to prevent it from stacking which effectively makes the player invincible.

Before Defending, DEF = 20:

```
============= ROUND 9 =============

Warrior HP: 65/260 | ATK: 40 | DEF: 20 | SPD: 30
Items: Potion, Potion
Special Skill Cooldown: 0 rounds

Goblin C HP: 5/55

--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item
4. Special Skill (Shield Bash)

Enter choice (1-4): 2
Warrior -> Defend: Defense increased by 10 for this round and the next.
```

After Defending, DEF = 20 + 10 = 30:

```
============== ROUND 10 ==============

Warrior HP: 60/260 | ATK: 40 | DEF: 30 | SPD: 30
Status: Defending (1 turns remaining)
Items: Potion, Potion
Special Skill Cooldown: 0 rounds

Goblin C HP: 5/55

--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item
4. Special Skill (Shield Bash)

Enter choice (1-4): 2
Warrior -> Defend: Defense increased by 10 for this round and the next.
```

Subsequent Consecutive Defends, DEF stays at 30:

```
============== ROUND 11 ==============

Warrior HP: 55/260 | ATK: 40 | DEF: 30 | SPD: 30
Status: Defending (1 turns remaining)
Items: Potion, Potion
Special Skill Cooldown: 0 rounds

Goblin C HP: 5/55

--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item
4. Special Skill (Shield Bash)

Enter choice (1-4): 2
Warrior -> Defend: Defense increased by 10 for this round and the next.
```

4. Replay Option

After a game ends, the user is able to replay the game without restarting the application

```
====================================================
All enemies defeated!
Remaining HP: 50/260
Total Rounds: 12
====================================================

1. Start a new game  2. Exit

Enter choice (1-2):
```

5. Input validation
Only valid user inputs are accepted, non valid inputs are re prompted.

```
--- Warrior's Turn ---
Choose action:
1. Basic Attack
2. Defend
3. Use Item
4. Special Skill (Shield Bash)

Enter choice (1-4): 5
Invalid input.

Enter choice (1-4):
```

1.4 Design Considerations

Our game design utilised the concepts of OOP that we have learnt from the course.

1.Encapsulation
We have implemented the combatants attributes, gameCLI helper functions and other important functions and methods as private, while others as public, with multiple getter and setter functions in order to help prevent any accidental modifications by other classes to critical aspects of the game, allowing the game to be consistent throughout.

Examples of encapsulation in the Combatant class:

```java
public abstract class Combatant {
    private String name;
    private int maxHP;
    private int currentHP;
    private int attack;
    private int defense;
    private int speed;
    private List<StatusEffect> statusEffects;
```

```java
public String getName() { return name; }
public int getMaxHP() { return maxHP; }
public int getCurrentHP() { return currentHP; }
public int getAttack() { return attack; }
public int getDefense() { return defense; }
public int getSpeed() { return speed; }
```

2.Inheritance

We especially utilised inheritance in our code to not only make our code cleaner, but it helps us to follow good coding principles especially in Open Closed principle, allowing us to extend our game by, for example, adding more player classes and types of enemies easily without modifying existing code.

Implementation of the goblin enemy type with minimal code:

```
public class Goblin extends Enemy {
    public Goblin(String name, EnemyActionStrategy strategy) {
        super(name, 55, 35, 15, 25, strategy);
    }
```

3.Polymorphism

We implemented polymorphism in our code with the main intention to ensure that our code does not repeat and to maintain simplicity, as shown especially in the execute method below:

Abstract execute method in interface class of Action:

```
public interface Action {
    void execute(Combatant actor, BattleContext context);
}
```

Implementation of execute in basicattack:

```
public void execute(Combatant actor, BattleContext context) {
    int damage = actor.getAttack() - target.getDefense();
    if (damage < 0) damage = 0;

    int hpBefore = target.getCurrentHP();
    target.takeDamage(damage);

    System.out.println(actor.getName() + " -> performs BasicAttack on-> " + target.getName()
            + ": HP: " + hpBefore + " -> " + target.getCurrentHP()
            + " (dmg: " + actor.getAttack() + "-" + target.getDefense() + "=" + damage + ")");

    if (!target.isAlive()) {
        System.out.println(target.getName() + " has been ELIMINATED!");
    }
}
```

Implementation of execute in useItem:

```
public void execute(Combatant actor, BattleContext context) {
    if (actor instanceof Player) {
        Player player = (Player) actor;
        System.out.println(actor.getName() + " -> Item -> " + item.getName() + " used");
        item.use(player, context);
        player.removeItem(item);
    }
}
```

4. Abstraction

We utilised Abstraction in our abstract and interface classes so that we can encourage low coupling between classes and to allow easy extensions for the future.

Example of abstraction by implementing a simple interface for subsequent new items to implement.

```
public interface Item {
    void use(Player player, BattleContext context);
    String getName();
}
```

2. Implementation of SOLID principles

We have designed and implemented our code closely following the SOLID principles in order to make our code maintainable, flexible and easy to understand.

1.Single Responsibility Principle

We intentionally designed classes so that it handles only one task.
Such as our GameCLI class, its responsibility is to handle the input from the user and provide the user the game's Output.
Another example would be that each subclass of Action only handles one type of action, such as basicAttack.java, it only handles basicattack from combatants.
All of this is to ensure code readability, simplicity and to prevent the creation of a God class.

Example of single responsibility in potion class, its responsibility is to heal the player by 100 hp.

```
public class Potion implements Item {

    public void use(Player player, BattleContext context) {
        int hpBefore = player.getCurrentHP();
        player.heal(100);
        System.out.println("Heal 100HP: HP: " + hpBefore + " -> " + player.getCurrentHP());
    }
```

2. Open Closed Principle
We followed this principle by creating abstract classes and interfaces that provide the main important contracts for concrete classes to implement so that we can easily implement new classes while preventing any existing code from being modified.

An example would be that if we wanted to add a new action such as Fireball, all we need to do is to implement the Action interface and work from there. No existing code modified.

```
public interface Action {
    void execute(Combatant actor, BattleContext context);
}
```

3. Liskov substitution Principle
We followed this principle in our design especially through the implementation of the Combatant abstract class and its subclasses, allowing us to preserve our code's functionality and its consistency without breaking it.

Example of LSP, where the game's combatant's turn sorting although accepts Combatant, we can input its subclasses such as Warrior or wolf confidently without breaking it.

```
public List<Combatant> determineTurnOrder(List<Combatant> combatants) {
    ArrayList<Combatant> sorted = new ArrayList<Combatant>(combatants);

    // bubble sort by speed (highest first)
    for (int i = 0; i < sorted.size() - 1; i++) {
        for (int j = 0; j < sorted.size()-i-1; j++) {
            if (sorted.get(j).getSpeed() < sorted.get(j+1).getSpeed()) {
                Combatant temp = sorted.get(j);
                sorted.set(j, sorted.get(j+1));
                sorted.set(j+1, temp);
            }
        }
    }
    return sorted;
}
```

4. Interface Segregation Principle
We followed this principle in our game design by intentionally implementing our interfaces and abstract classes to hold not only the minimum number of methods to be implemented, but the

core and important ones as well. This is done to promote low coupling and readability in our code.

Example of an interface following ISP

```
public interface Item {
    void use(Player player, BattleContext context);
    String getName();
}
```

Subsequent implementation of Item interface for Potion class:

```
public class Potion implements Item {

    public void use(Player player, BattleContext context) {
        int hpBefore = player.getCurrentHP();
        player.heal(100);
        System.out.println("Heal 100HP: HP: " + hpBefore + " -> " + player.getCurrentHP());
    }
```

5. Dependency Inversion Principle

We followed this principle by implementing our high level class to not depend on a concrete class, but rather, our abstract or interface class.

Example showing that Enemy class depends on interface EnemyActionStrategy and not the concrete class BasicAttackStrategy.

```
public abstract class Enemy extends Combatant {
    private EnemyActionStrategy actionStrategy;

    public Enemy(String name, int maxHP, int attack, int defense, int speed, EnemyActionStrategy strategy) {
        super(name, maxHP, attack, defense, speed);
        actionStrategy = strategy;
    }

    public EnemyActionStrategy getActionStrategy() { return actionStrategy; }
```

3. Trade-Offs

1. Our decision to separate all of the actions away from the main Action file into its own separate files, and other areas such as statuseffects, items and combatants has undoubtedly created a lot more files to handle and maintain. But this decision was guided so as to follow the SOLID principles, especially single responsibility and Open/closed principle. Thus although there are a lot more files, this drastically improves extensibility and minimises any ripple effects if a change has to be made, improving our code's maintainability.

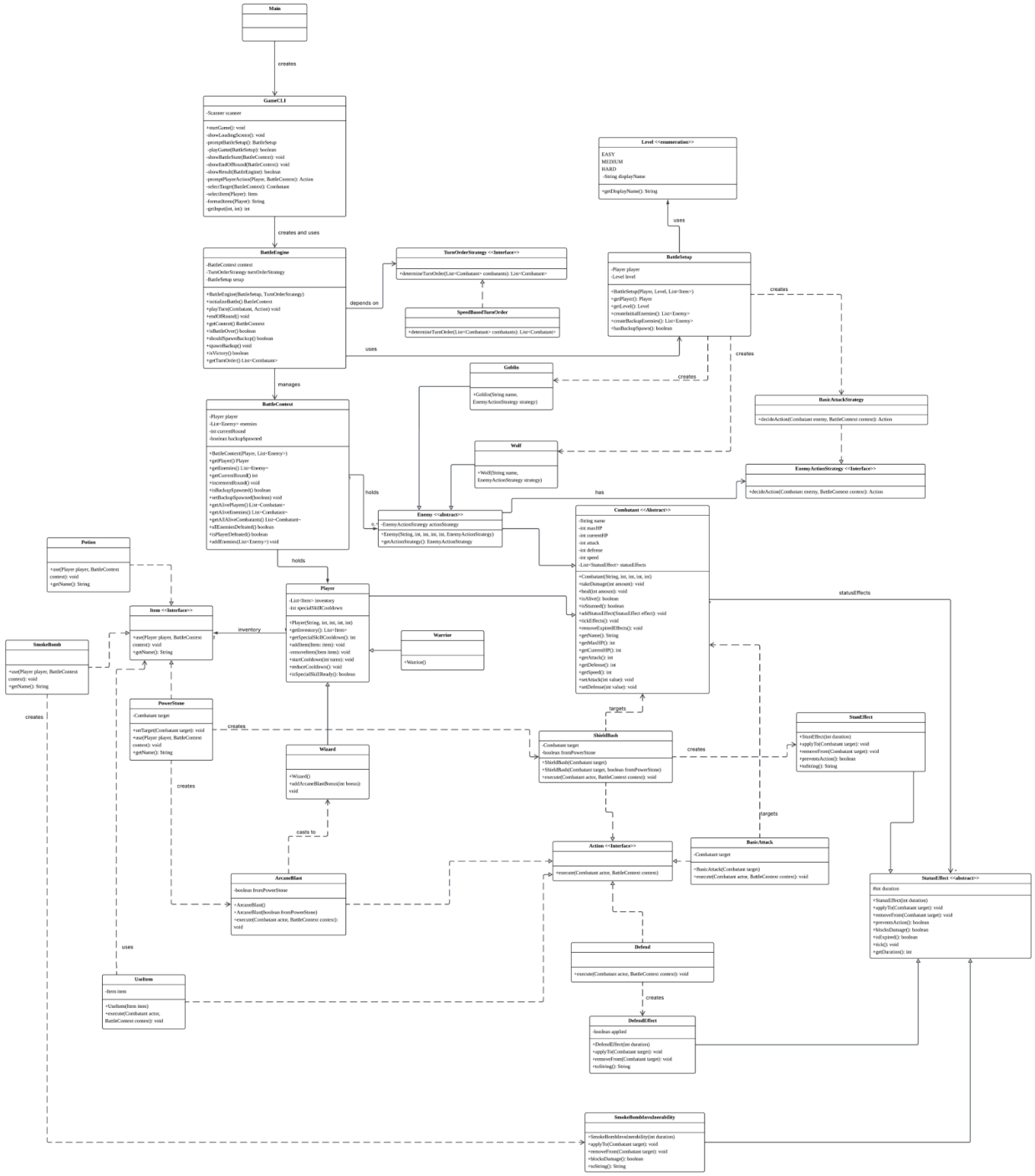
2. To ensure that we followed through SOLID principles, more specifically the single responsibility principle, we had to push all of the methods that handle user input, UI output etc, to a single class, GameCLI. Which has made the class file very large, being over 330+ lines of code.

3.1 Design Patterns

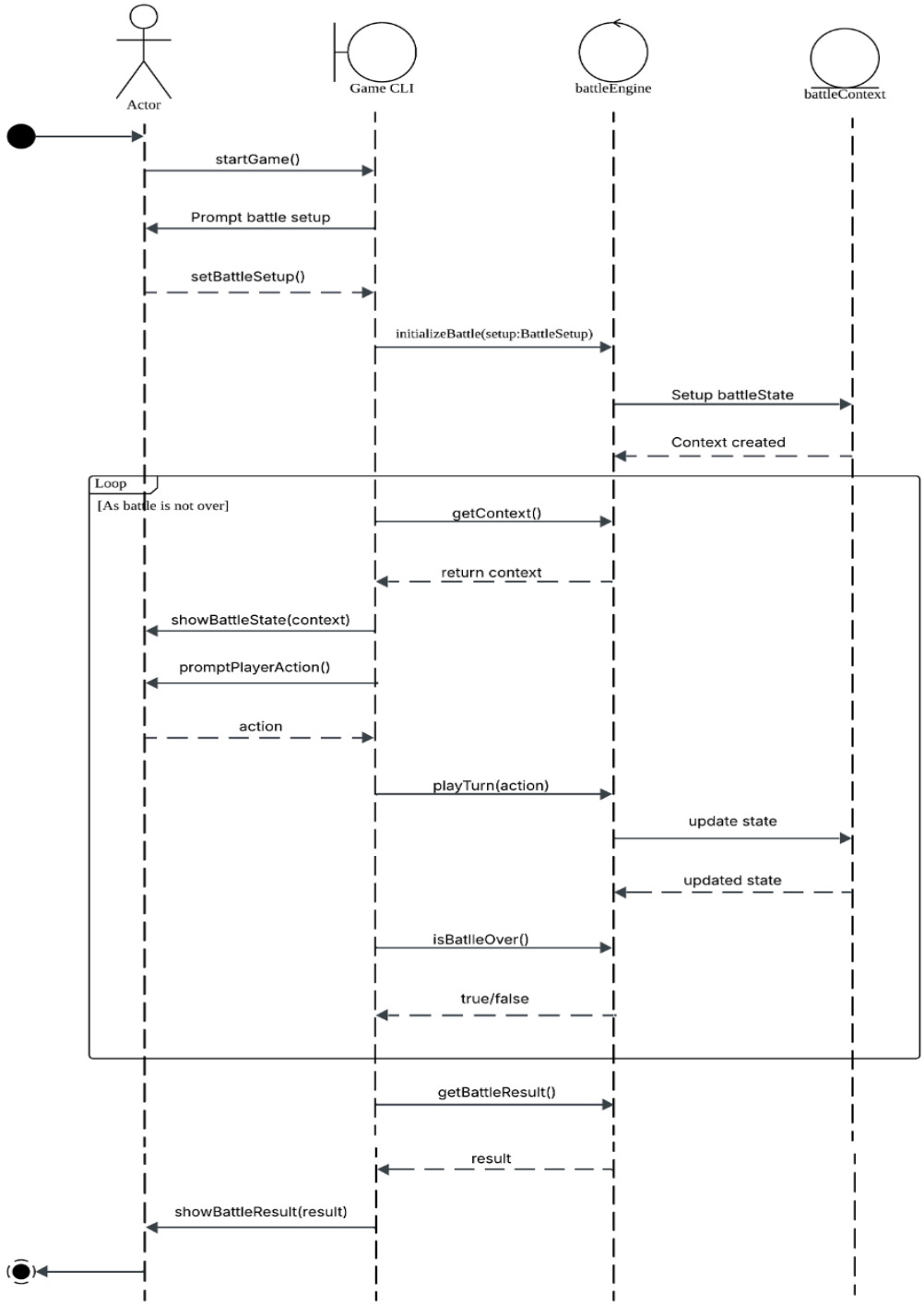
We mainly utilised Strategy Pattern in our game design
An example would be in our TurnOrderStrategy and SpeedBasedTurnOrder. Right now the combatants are sorted by their speed in SpeedBasedTurnOrder. In the future if the game wants another way of sorting the combatants etc, all we need to do is to just create a new file and implement our algorithm in it, never needing to touch any of the engine etc.

So why did we choose this pattern? It is because we want to enforce the open closed principle in our game design. By separating our turn order algorithm away from the battle engine, the system remains open to extensions but closed for modifications. Allowing our game to be more easily maintained and easier to extend in the future.

4. UML Class Diagram



5. UML Sequence Diagram



5.1 Gameplay Logic Flow

The following scenarios were chosen because they are among the most common in our game and best represent the core interactions between the methods in our game design.

Example scenario: Goblin attacks Warrior

- **Goblin's turn begins.**
- GameCLI calls goblin.getActionStrategy().
- Goblin returns its strategy: BasicAttackStrategy.
- GameCLI calls decideAction() on BasicAttackStrategy.
- BasicAttackStrategy creates a new BasicAttack targeting the Warrior.
- BasicAttackStrategy returns the BasicAttack action.
- GameCLI calls BattleEngine.playTurn(goblin, basicAttackAction).
- Inside playTurn(), BattleEngine calls goblin.isAlive().
- Goblin returns true.
- BattleEngine calls goblin.isStunned().
- Goblin checks its status effects for stun.
- No stun effect is found, so isStunned() returns false.
- Since the Goblin is alive and not stunned, BattleEngine calls basicAttackAction.execute(goblin,context)
- Inside BasicAttack.execute():
- goblin.getAttack() returns 35.
- warrior.getDefense() returns 20.
- Damage is calculated as 35 - 20 = 15.
- BasicAttack calls warrior.takeDamage(15).
- Inside takeDamage():
- Warrior checks its status effects by calling blocksDamage() on each.
- None block the damage, which means no smokebomb, so warrior takes full damage.
- Warrior.currentHP decreases from 260 to 245.
- BasicAttack prints the combat log:
- "Goblin -> BasicAttack -> Warrior: HP: 260 -> 245 (dmg: 35-20=15)"

Example Scenario 2: Warrior basicAttacks goblin

- **Warrior's turn begins.**
- GameCLI calls warrior.reduceCooldown().
- specialSkillCooldown is reduced from 2 to 1.
- GameCLI displays the action menu.
- The player selects Basic Attack.
- GameCLI checks for available enemy targets.
- Since Goblin is the only alive enemy, it is auto-selected.
- GameCLI prints "Target: Goblin".
- GameCLI creates a new BasicAttack(goblin).
- GameCLI calls BattleEngine.playTurn(warrior, basicAttackAction).
- Inside playTurn(), BattleEngine calls warrior.isAlive().
- Warrior returns true.
- BattleEngine calls warrior.isStunned().
- Warrior checks its status effects for stun.
- No stun effect is found, so isStunned() returns false.
- Since the Warrior is alive and not stunned, BattleEngine calls basicAttackAction.execute(warrior, context).
- Inside BasicAttack.execute():
- warrior.getAttack() returns 40.
- goblin.getDefense() returns 15.
- Damage is calculated as 40 - 15 = 25.
- BasicAttack calls goblin.takeDamage(25).
- Inside takeDamage():
- Goblin checks its status effects by calling blocksDamage() on each.
- None block the damage.
- Full damage is applied.
- Goblin.currentHP decreases from 30 to 5.
- BasicAttack prints the combat log:
- "Warrior -> BasicAttack -> Goblin: HP: 30 -> 5 (dmg: 40-15=25)"
- BasicAttack calls goblin.isAlive().
- Goblin still has 5 HP, so it remains alive.
- The Goblin is not eliminated.

Code Snippet for both scenarios:

```
if (combatant instanceof Player) {
    Player p = (Player) combatant;
    p.reduceCooldown();
    Action action = promptPlayerAction(p, context);
    engine.playTurn(combatant, action);
} else if (combatant instanceof Enemy) {
    Enemy enemy = (Enemy) combatant;
    Action action = enemy.getActionStrategy().decideAction(enemy, context);
    engine.playTurn(combatant, action);
}
```

6. Reflection

We have encountered multiple difficulties along the way of designing this game.

1.Compatibility Issues

The biggest difficulty we had was to make all of our classes compatible with each other. Because we approached this project from the bottom up, where we developed our interfaces and abstract classes first, then its subclasses, each class was developed not only modularly, but independently as well. Although this allowed us to design reusable classes with low coupling in mind, this resulted in multiple compatibility issues, mismatched methods, dependency conflicts and errors when tying it all together.

How we overcame this obstacle and what we learnt from this is to take smaller steps when trying to connect it all together. Instead of trying to connect the entire system at once, we tested smaller flows first such as action class execution, status effect application before combining it. This made it easier for us as it allowed us to isolate where in the code the bugs were happening and helped us to fix them one at a time.

From this, we have learnt that good software design is not only about creating low coupling and extensible classes, but it is ensuring that they work together to form a complete system.

2.Status effect ticking.

We faced constant issues in trying to find out why status effects are ticking so fast, especially after each turn, a skill that was on cooldown with 3 turns would suddenly become available when the user is on their turn again. We realised the issue was that we implemented the decrementation of the cooldown in the wrong location, where we should have placed it at the end of each round instead of each combatant's turn. Thus we fixed it by implementing endOfRound() at the end of the battle loop logic that iterates through every combatant's statuseffects and decrements it to create an accurate cooldown counter.
From this, we learnt the importance of carefully defining when state changes should happen in a turn based system. Even if the logic itself is correct, placing it in the wrong part of the control flow can cause major gameplay issues

3.Defense Stacking Bug

During game testing, we have discovered a game breaking bug which allowed players to consecutively stack their defense which effectively makes them immortal. We have thought of multiple solutions such as adding flags to prevent defense etc, many of the approaches increase coupling dramatically which we did not want. In the end, we chose to do an iteration check where before adding a defense, we iterate through the combatant's statuseffect list to find out if it already has that effect. If it does, then we break without adding the effect, effectively wasting the user's turn. We chose this approach because it helps to maintain low coupling as we didn't need to implement any new logic just for this, in the battleengine or anywhere else.
From this, we have learnt that solving bugs is not just about making the program work, but also about choosing solutions that remain consistent with the overall design principles of the system.

7. Declarations

Appendix B

<u>Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment</u>

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
Zhong Xingyang	SC2002	FAIA Group 6	16/04/2026 Zhong Xingyang
Yang Siqiao	sc2002	FAIA Group 6	16/04/2026 Yang Siqiao
Yee Woon Chun Gerrard	SC2002	FAIA Group 6	16/04/2026
Yang Danni	SC2002	FAIA Group 6	16/04/2026

Important notes:

1. Name must **EXACTLY MATCH** the one printed on your Matriculation Card.

2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

Appendix C

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I _____Zhong Xingyang_____ (student name), _A250048_____@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming

2. Course Code: SC2002

3. Instructor: Li Fang

4. Title of Assignment/Project Submission: Turn Based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only.

☐

ii. ii. Used GAI as permitted to assist in generating a first text

only. ☐

And/or iii. Used GAI to refine syntax and grammar for correct language

submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

_Zhong Xingyang_____

Zhong Xingyang _____16/04/2026_____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, Yee Woon Chun Gerrard (student name), gyee005@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming

2. Course Code: SC2002

3. Instructor: Dr Li Fang

4. Title of Assignment/Project Submission: Turn Based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Student Name & Signature Date :

Yee Woon Chun Gerrard, 16 April 2026

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Yang Siqiao (student name), YANG0966 @e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming

2. Course Code: SC2002

3. Instructor: Dr Li Fang

4. Title of Assignment/Project Submission: Turn Based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

SC2002 Group Assignment AY25/26 Semester 2

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Yang Siqiao

16/04/2026 YangSiqiao Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

SC2002 Group Assignment AY25/26 Semester 2

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I Yang Danni (student name), YANG1051 @e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: Object Oriented Design and Programming

2. Course Code: SC2002

3. Instructor: Dr Li Fang

4. Title of Assignment/Project Submission: Turn Based Combat Assignment

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

SC2002 Group Assignment AY25/26 Semester 2

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Yang Danni

16/04/2026 Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.